

Assignment 2: Email Threat Detection

Item	Requirement
Submission file	Assignment2-LastName.ipynb
Dataset	Enron-Spam Dataset
Required models	Naive Bayes, SVM, MLPClassifier, BERT component
AI use	Allowed with required AI Use Statement
Total points	100

Introduction

In Lab 2, you completed a guided text classification workflow using the SMS Spam Collection dataset. In this assignment, you will extend that workflow into a more complete email threat detection analysis using the Enron-Spam Dataset.

You will inspect the dataset, construct a model-ready email text field, identify possible leakage, train and compare multiple text classification models, inspect model mistakes, and explain the results from a cybersecurity perspective. This assignment focuses on spam detection.

Dataset

Dataset	Required File	Text Columns	Target Column	Class Meaning
Enron-Spam Dataset	Enron_Spam_Dataset.csv	Subject and Message	Spam/Ham	ham = legitimate email spam = spam email

Important dataset rules:

- Treat spam as the positive class for precision, recall, and F1-score.
- Use Subject and Message as text input features.
- Do not use Message ID as a model feature.
- Do not use Date as a model feature. Date may create artificial performance because it can reveal dataset collection patterns rather than email content.
- Use Date only for exploratory analysis and leakage discussion.

Security and Privacy Note

Enron emails are public research data, but they still come from real workplace communications. Treat the data responsibly.

- Do not paste full email messages into your written explanations.
- When showing examples of model mistakes, show only short, sanitized excerpts.

- Remove or mask email addresses, names, phone numbers, or other personal details if they appear in excerpts.
- Your goal is defensive model evaluation, not exposing private or embarrassing content.

AI Use Policy

You may use AI tools such as ChatGPT, Copilot, Gemini, Claude, or similar tools. However, you are responsible for understanding, testing, correcting, and explaining everything you submit.

Allowed AI use:

- Explaining code errors
- Suggesting debugging strategies
- Explaining Python, Pandas, scikit-learn, or transformer-related functions
- Suggesting text preprocessing ideas
- Helping interpret model evaluation results
- Improving grammar and clarity in written explanations

Not allowed:

- Submitting AI-generated work that you do not understand
- Submitting code that you cannot explain
- Fabricating results
- Claiming a model performed better than it actually did
- Claiming the model detects phishing when the training label is only ham or spam
- Copying another student's notebook or report

At the end of your notebook, include an AI Use Statement. Use one of the following formats:

```
AI Use Statement: I used [tool name] to help with [specific task]. I reviewed, tested,
and modified the output before including it in my assignment.
AI Use Statement: I did not use AI tools for this assignment.
```

Required Packages

Use the following packages for the classical machine learning parts. You may import additional packages if needed.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.model_selection import GridSearchCV
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC, SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
```

For the BERT component, use Google Colab.

```
import torch
from transformers import AutoTokenizer, AutoModel
# Optional for fine-tuning:
# from transformers import AutoModelForSequenceClassification, Trainer,
# TrainingArguments
```

Part 1: Dataset Loading and Inspection

Load the Enron-Spam dataset and inspect its structure before doing any modeling.

1. Load *Enron_Spam_Dataset.csv*.
2. Display the first five rows.
3. Display the number of rows and columns.
4. Display column names and data types.
5. Check missing values by column.
6. Check exact duplicate rows.
7. Display class counts and class percentages for Spam/Ham.

Required outputs:

- First five rows
- Shape of the dataset
- Column names and data types
- Missing value summary
- Duplicate row count
- Class distribution table

Part 2: Construct the Email Text Field

Create one model input column by combining Subject and Message. This combined text field should be used for all text classification models.

1. Replace missing Subject values with an empty string.
2. Replace missing Message values with an empty string.
3. Create a new column named text by combining Subject and Message.
4. Create a short summary of empty or near-empty text records.
5. Remove records where the combined text is empty after stripping whitespace.
6. Report duplicate combined text records and decide whether to remove them. If you remove duplicates, report the new class distribution.

Required outputs:

- Preview of Subject, Message, text, and Spam/Ham
- Number of empty combined text records
- Number of duplicate combined text records
- Final row count after cleaning
- Class distribution after cleaning

Part 3: Class Distribution and Text Length Analysis

Explore whether the classes and message lengths differ between ham and spam.

1. Create a class distribution plot.
2. Create character length and word count features for the combined text.
3. Report mean and median character length by class.
4. Report mean and median word count by class.
5. Create at least two text length visualizations, such as histograms or boxplots.
6. Briefly explain whether the dataset is balanced and whether text length appears different across classes.

Required outputs:

- Class distribution plot
- Text length summary table by class
- At least two text length visualizations
- Short interpretation of class balance and text length patterns

Part 4: Leakage and Dataset Caveat Analysis

Before training models, inspect whether non-content fields may accidentally reveal the label.

1. Analyze the distribution of Date by Spam/Ham.
2. Explain why Date should not be used as a model feature.
3. Explain why Message ID should not be used as a model feature.
4. State clearly which columns will be excluded from modeling.
5. State clearly that the model learns ham versus spam, not phishing versus safe.

Required outputs:

- Date distribution summary or visualization by class
- List of excluded columns
- Short leakage explanation
- Short dataset caveat statement

Part 5: Train-Test Split and TF-IDF Preparation

Prepare the training and testing data for text classification.

1. Create X from the cleaned text column.
2. Create y from the Spam/Ham target column.
3. Split the data into training and testing sets using stratification.
4. Use a 75 percent training and 25 percent testing split.
5. Use *random_state=40103*.
6. Fit TF-IDF only on the training text, not on the full dataset.
7. Use the same fitted TF-IDF vectorizer to transform the test text.

You may use scikit-learn Pipeline objects for this assignment. Pipelines are recommended because they help prevent preprocessing leakage.

Required outputs:

- Training and testing row counts
- Training and testing class proportions
- TF-IDF vocabulary size or feature matrix shape
- Brief explanation of why the vectorizer should be fit only on training data

Part 6: Baseline Text Classification Models

Train and evaluate the two main classical text classifiers from this module.

1. Train a Multinomial Naive Bayes model using TF-IDF or count-based text features.
2. Train a Linear SVM model using TF-IDF features.
3. Evaluate both models on the test set.
4. For each model, generate a classification report and confusion matrix.
5. Create a metric summary table using accuracy, precision, recall, and F1-score.

Model	Accuracy	Precision	Recall	F1
Naive Bayes				
Linear SVM				

Required outputs:

- Classification report for Naive Bayes
- Confusion matrix for Naive Bayes
- Classification report for Linear SVM
- Confusion matrix for Linear SVM
- Metric summary table

Part 7: Stratified k-Fold Cross-Validation

Use stratified 5-fold cross-validation to compare Naive Bayes and Linear SVM more reliably.

1. Use **StratifiedKFold** with **n_splits=5**, **shuffle=True**, and **random_state=40103**.
2. Evaluate Naive Bayes and Linear SVM using accuracy, precision, recall, and F1-score.
3. Report mean performance across folds.
4. Briefly compare cross-validation results with test-set results.

Model	CV Accuracy	CV Precision	CV Recall	CV F1
Naive Bayes				
Linear SVM				

Briefly explain why stratified cross-validation is useful for text-based security datasets.

Part 8: SVM Variant Sample Experiment

Kernel SVMs can be expensive on large high-dimensional text data. For this reason, run SVM variant comparisons on a smaller stratified sample.

1. Create a stratified sample of no more than 5,000 records.
2. Use a controlled TF-IDF setting, such as a limited **max_features** value.
3. Train and evaluate the following: Linear SVM, RBF SVM, Polynomial SVM, Sigmoid SVM.
4. Report test-set metrics for the sampled experiment.
5. Explain which SVM variant performed best and whether the extra cost was justified.

SVM Variant	Sample Size	Accuracy	Precision	Recall	F1	Comment
Linear SVM						
RBF SVM						
Polynomial SVM						
Sigmoid SVM						

Required outputs:

- Sample size and class distribution
- SVM variant comparison table
- Short explanation of performance versus computational cost

Part 9: MLPClassifier Experiment

Train one Multi-Layer Perceptron model using TF-IDF features. Keep this experiment controlled and explain your choices.

1. Use a limited TF-IDF feature space so the MLP experiment can run in a reasonable time.
2. Train one MLPClassifier model with one or two hidden layers.
3. Report the model settings you used, such as **hidden_layer_sizes**, **activation**, **max_iter**, and **random_state**.
4. Evaluate the model using a classification report and confusion matrix.
5. Compare MLP performance against Naive Bayes and Linear SVM.

Required outputs:

- MLP settings table or short description
- MLP classification report
- MLP confusion matrix
- Short comparison with Naive Bayes and Linear SVM

Part 10: BERT Component

Complete a BERT-based text representation or classification experiment in Google Colab.

Choose one of the following BERT paths:

- Path A: Use a pretrained BERT tokenizer and model to create embeddings, then train a simple classifier on the embeddings.
- Path B: Fine-tune **AutoModelForSequenceClassification** on a small, stratified sample.

You may use a small, stratified sample because BERT can be slow. Report the sample size clearly.

1. State the BERT model name you used.
2. State the sample size and class distribution.
3. Show that the tokenizer runs on example emails.
4. Train or apply the BERT-based model or embedding workflow.
5. Evaluate the BERT component using accuracy, precision, recall, F1-score, and confusion matrix.
6. Explain whether BERT improved the decision enough to justify the added complexity.

Required outputs:

- BERT model name
- BERT sample size and class distribution
- Tokenizer output for one short example
- BERT-based evaluation results

Part 11: Final Model Comparison

Create one final comparison table and inspect model mistakes from a security perspective.

Model	Data Used	Accuracy	Precision	Recall	F1	Main Strength	Main Weakness
Naive Bayes	Full dataset						
Linear SVM	Full dataset						
Best SVM Variant	Sample						
MLPClassifier	Full or limited features						
BERT Component	Sample						

1. Identify the best overall model based on F1-score and security usefulness.
2. Identify the model with the best spam recall.
3. Inspect at least five false positives if available.
4. Inspect at least five false negatives if available.

5. Show only short sanitized excerpts of emails, not full messages.
6. Explain whether false positives or false negatives are more harmful in spam filtering.

Required outputs:

- Final model comparison table
- False positive examples or explanation if none are available
- False negative examples or explanation if none are available
- Short security interpretation of the most important mistakes

Part 12: Deployment Recommendation

Write a short deployment recommendation based on your actual results. Your explanation should be clear, specific, and connected to the metrics and confusion matrices shown in your notebook.

1. State which model you would choose.
2. State whether the model should send emails to inbox, warning, quarantine, or human review.
3. Explain the tradeoff between spam recall and ham false positives.

Part 13: AI Use Statement

End your notebook with a clear AI Use Statement. State which tool you used, what you used it for, and how you verified or modified the output. If you did not use AI, state that directly.

What to Submit

Submit one Notebook file only:

```
Assignment2-LastName.ipynb
```

The notebook must include code, outputs, plots, tables, written explanations, sanitized example excerpts, and the AI Use Statement. No separate PDF is required.

Grading Rubric

Component	Points
Dataset loading and inspection	10
Text construction, cleaning, and privacy controls	10
Class distribution, text length analysis, and leakage discussion	10
Train-test split and TF-IDF preparation	10
Naive Bayes and Linear SVM evaluation	15
Stratified k-fold cross-validation	10
SVM variant sample experiment	10
MLPClassifier experiment	10
BERT component	10
Final comparison, error analysis, deployment recommendation, and AI Use Statement	5
Total	100

Important Notes

- Do not use Message ID or Date as model features.
- Do not fit TF-IDF, scaling, encoding, or any other preprocessing step on the full dataset before splitting.
- Do not report only accuracy.
- Precision, recall, F1-score, and confusion matrix must all be considered.
- A model with high accuracy but low spam recall may still be a weak email threat detector.
- This is a ham versus spam classifier, not a true phishing detector.
- Your explanations must be written in your own words.
- You must be able to explain your code if asked.